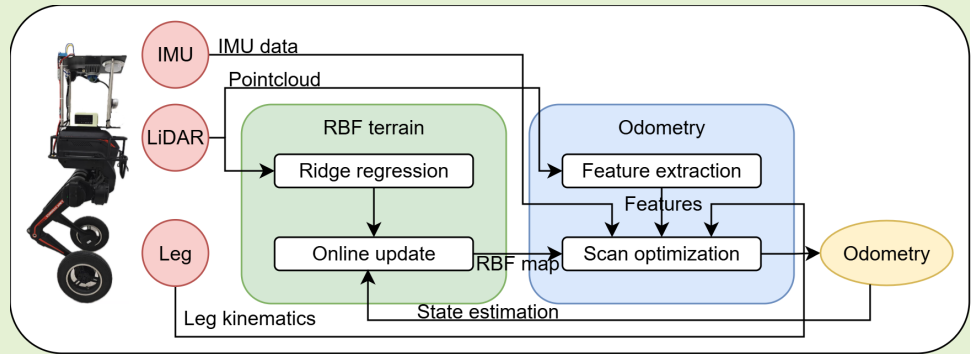


LiDAR-Inertial Odometry via Online RBF Terrain Surface Modeling for Legged-Wheel Robots

Yizhe Liu, Chang Shen, Han Zhang, Jingchuan Wang, Weidong Chen

Abstract—Legged-wheel robots excel at traversing unstructured terrains, including bumpy roads and staircases. However, in such environments, neglecting terrain geometry can lead to severe odometry drift. In particular, abrupt vertical accelerations caused by uneven surfaces may introduce significant pose errors along the z-axis, thereby degrading both localization and mapping accuracy. To address this issue, we propose a terrain-aware LiDAR-inertial odometry (LIO) framework that approximates the terrain using Radial Basis Functions (RBFs) whose centers are adaptively selected and weights are recursively updated. The resulting smooth terrain manifold enables “soft constraints” that regularize the odometry optimization and mitigate the z-axis pose drift under abrupt elevation changes during robot maneuvers. To ensure real-time performance, the RBF-related operations are accelerated on the GPU. Experiments on unstructured terrains demonstrate that the proposed method improves localization accuracy and map consistency, especially in scenarios with abrupt elevation changes and sparse geometric features.

Index Terms—Simultaneous localization and mapping, terrain mapping, LiDAR-inertial odometry (LIO), legged-wheel robots



I. INTRODUCTION

LIDAR-INERTIAL odometry (LIO) relies on geometrically informative LiDAR measurements to correct inertial drift. Legged-wheel robots combine the high mobility of wheeled platforms with the terrain adaptability of legged systems, making them well-suited for inspection tasks in complex environments. In practice, they are often required to traverse challenging terrains such as staircases and bumpy roads. When these robots traverse uneven terrain, abrupt vertical motion and rapid attitude variations make inertial propagation more susceptible to bias and integration errors, while LiDAR scan matching may provide insufficient geometric correction in the vertical direction. Consequently, these errors can accumulate into substantial pose drift, particularly along the z-axis.

The physical wheel-terrain contact provides an additional geometric relation between the robot pose and the terrain, especially for height and attitude. General-purpose LIO methods, such as LIO-SAM [1], Fast-LIO2 [2], [3], and GLIM [4], improve LiDAR-inertial estimation but do not explicitly

exploit this relation. Consequently, their standard formulations do not use wheel-terrain contact to correct residual height and attitude errors.

To exploit this relation for both height and attitude correction as contact locations change, the terrain representation should provide height and normal at each contact location. Grid-based elevation-mapping approaches [5]–[7] represent terrain on fixed-resolution 2.5-D grids, so the accuracy of height and derived-normal queries is tied to the grid discretization. LIO-GC [8] extracts ground points using optimized Cloth Simulation Filtering. The ground points are then organized into a smoothed grid-based slope range map, and the resulting ground factors are incorporated into the LIO-SAM back end to improve vertical accuracy. These approaches provide useful terrain information, but their standard formulations do not evaluate wheel-contact constraints from an explicit differentiable terrain function during pose optimization. Regression can instead produce a differentiable terrain function that provides height and normal directly. However, repeatedly fitting such a function in batch from all accumulated observations causes the regression problem to grow with the observation history. Per-cycle refitting is therefore impractical for online LIO.

To address these challenges, we propose RBF-LIO, a terrain-aware representation-update-optimization framework.

Yizhe Liu and Chang Shen contribute equally to this work. All the authors are with School of Automation and Intelligent Sensing, Shanghai Jiao Tong University, Institute of Medical Robotics, Shanghai Jiao Tong University, and Key Laboratory of System Control and Information Processing, Ministry of Education, Shanghai 200240, China. Corresponding author: Han Zhang (e-mail: zhanghan_tc@sjtu.edu.cn)

RBF-LIO represents terrain as a continuous and differentiable weighted sum of radial basis functions (RBFs). The surface provides height and normal at every wheel contact; recursive ridge regression carries terrain information across frames while assimilating only new observations; and manifold residuals incorporate these quantities into LIO scan matching. Because all per-wheel residuals use the same terrain surface and constrain the same base pose, terrain information from different contact locations jointly constrains height and attitude. The framework-level contribution lies in the unified coupling of terrain representation, recursive updating, and contact-constrained pose optimization. More specifically, the main contributions of this work are threefold:

- We develop an RBF terrain model whose weights are updated online through recursive ridge regression. Adaptive center selection retains only candidates with sufficient LiDAR support, while GPU parallelization accelerates the RBF computations.
- We formulate wheel–terrain manifold residuals that couple the differentiable RBF surface with LIO scan matching. These residuals enforce terrain contact, wheel geometry, and alignment between the wheel radial direction and the terrain normal to suppress height and attitude drift.
- We demonstrate improved accuracy and map consistency through quantitative experiments on the Tron1 platform and further show cross-platform operation in a qualitative staircase test on the DeepRobotics Lynx M20 with a different LiDAR mounting configuration. We also release the collected dataset¹ for future evaluation of SLAM methods for legged-wheel robots.

II. RELATED WORK

To improve state estimation on legged or wheel-legged platforms, several works exploit kinematic or contact information. Leg-KILO [9] combines kinematic–inertial and LiDAR odometry for dynamic legged robots. Its kinematic–inertial estimator queries LiDAR map points near each stance foot, fits a local plane, and uses the projected foot z -coordinate as a scalar contact-height measurement in the on-manifold ESKE. Wheel-Legged SLAM [10] instead introduces a kinematics-derived vertical correction factor and a gravity re-estimation factor into its factor-graph back end. VILENS [11] tightly couples preintegrated leg odometry with visual, inertial, and LiDAR measurements, but its odometry formulation does not derive contact constraints from an explicit terrain surface. LIKO [12] uses joint encoders and force/torque sensing to form kinematic and contact measurements while estimating the robot state and foot-contact positions online. These methods show the value of kinematic and contact information, but none evaluates terrain height and normal at all contact points from a single differentiable nonplanar terrain function. Consequently, they do not couple terrain constraints at different contact locations through one surface geometry, which limits the joint correction of height and attitude drift on uneven terrain.

Other SLAM systems address localization for ground vehicles or quadruped robots without contact sensing. ROLO-SLAM [13] mitigates vertical drift by decoupling rotation and translation in its LiDAR-only front end: forward location prediction provides a coarse translation estimate, spherical alignment estimates rotation, and geometric and continuous-time terms refine translation. ARS-SLAM [14] combines a FAST-LIO2-based front end with Cartographer and uses a pose-threshold strategy to reduce redundant loop-detection candidates during large-scale quadruped mapping, while FRL-SLAM [15] uses sliding-window optimization, online gravity-vector rectification, and CUDA acceleration for quadruped navigation. These methods improve pose estimation or system efficiency without an explicit terrain surface. They therefore do not provide an additional terrain-contact constraint when residual height and attitude errors remain.

For continuous terrain modeling, Leininger et al. [16] use an inducing-point sparse Gaussian process (SGP) to estimate local elevation, uncertainty, and gradients. The released implementation fits a new SGP from the current point cloud for each local-map request instead of maintaining it recursively across updates, and the resulting terrain model supports traversability planning rather than odometry optimization.

Motivated by these limitations, RBF-LIO evaluates height and normal constraints for all wheel contacts using the same online RBF surface and incorporates the resulting residuals into scan matching. The surface is updated recursively with new LiDAR observations instead of being repeatedly refitted in batch. This coupling of continuous terrain representation, recursive updating, and contact-constrained pose optimization constitutes the framework-level contribution of RBF-LIO.

III. PRELIMINARIES AND PROBLEM FORMULATION

To proceed, we first formulate the terrain-aware LIO problem considered in this work. This section shall clarify the role of a continuous and differentiable terrain representation and how the terrain approximation model enters the scan-matching objective.

At the k -th LiDAR frame, the robot base pose is denoted by

$$\eta_k = ({}^0\mathbf{R}_{B,k}, {}^0\mathbf{t}_{B,k}), \quad (1)$$

where ${}^0\mathbf{R}_{B,k} \in \text{SO}(3)$ and ${}^0\mathbf{t}_{B,k} \in \mathbb{R}^3$ denote the rotation and translation from the robot base frame to the world frame, respectively. A conventional LIO system combines inertial propagation with LiDAR scan matching. Since the proposed terrain constraints augment the scan-matching stage, we write the corresponding pose-estimation problem as

$$\min_{\eta_k} J_{\text{scan}}(\eta_k), \quad (2)$$

where J_{scan} denotes the LiDAR scan-matching cost of the underlying LIO system.

One class of terrain representations uses fixed-resolution 2.5-D elevation grids [5]–[7]:

$$p_z = h_{\text{grid}}(i, j), \quad (3)$$

where $i, j \in \mathbb{Z}$ are the indices of a fixed-resolution grid and p_z is the stored terrain height. The accuracy of height and

¹Available at https://zhanghan-tc.github.io/legged_wheel_dataset/

derived-normal queries is tied to the grid resolution. Moreover, since h_{grid} is defined only at discrete grid locations, the grid itself does not directly provide the analytic spatial derivatives needed to construct contact-constraint Jacobians at continuous horizontal coordinates. Thus, additional processing is required before grid-based terrain information can be used in iterative pose optimization.

To incorporate terrain geometry into state estimation, a contact constraint must relate each contact point to the local terrain. Leg-KILO [9], for example, fits a plane to LiDAR map points near each stance foot and uses the fitted plane to construct a contact-height constraint. The local plane is expressed as

$$\mathbf{n}^\top (\boldsymbol{\chi} - \mathbf{p}_0) = 0, \quad (4)$$

where $\boldsymbol{\chi}$ is a terrain contact point, $\mathbf{n}$ and $\mathbf{p}_0$ denote the local plane normal and a point on the plane, respectively. However, such an approximation is valid only within a limited neighborhood. Near stair edges, curved slopes, or transitions between different terrain surfaces, the fitted plane may vary abruptly and does not enforce a spatially consistent terrain function across contact locations.

The limitations of fixed grids and local planes motivate a continuous and differentiable terrain representation. We therefore model the terrain within a local region of interest as a locally single-valued height function

$$p_z = f(\mathbf{x}; \mathbf{w}), \quad \mathbf{x} = [p_x \ p_y]^\top, \quad (5)$$

where $\mathbf{w}$ denotes the parameters that characterize the terrain model. Such a representation provides terrain-height queries at arbitrary horizontal locations, admits spatial differentiation, and supports online updates from streaming LiDAR measurements. These properties allow the terrain geometry to enter LIO as an additional pose constraint.

To this end, let $\mathcal{W}$ denote the platform-specific set of wheels whose contact constraints are included in the estimator. Each wheel $j \in \mathcal{W}$ has an effective contact point $\boldsymbol{\chi}_{j,k} \in \mathbb{R}^3$, and the contact variables are stacked into a column vector as

$$\boldsymbol{\chi}_k = \text{col}_{j \in \mathcal{W}} (\boldsymbol{\chi}_{j,k}) \in \mathbb{R}^{3|\mathcal{W}|}. \quad (6)$$

For the two legged-wheel platform, $\mathcal{W} := \{\text{L}, \text{R}\}$, whereas the four legged-wheel platform uses $\mathcal{W} := \{\text{FL}, \text{FR}, \text{RL}, \text{RR}\}$. Accordingly, the terrain-aware estimation problem is written as

$$\min_{\boldsymbol{\eta}_k, \boldsymbol{\chi}_k} J_{\text{scan}}(\boldsymbol{\eta}_k) + J_{\mathcal{M}}(\boldsymbol{\eta}_k, \boldsymbol{\chi}_k; \mathbf{f}_k, \mathbf{q}_k), \quad (7)$$

where $f_k(\mathbf{x}) = f(\mathbf{x}; \mathbf{w}_k)$ denotes the terrain estimate at frame k , $\mathbf{q}_k$ denotes the measured joint configuration, and $J_{\mathcal{M}}$ aggregates the manifold residuals associated with all $j \in \mathcal{W}$. The terrain update and pose optimization are executed sequentially: once f_k is updated, f_k and $\mathbf{q}_k$ are held fixed while $\boldsymbol{\eta}_k$ and $\boldsymbol{\chi}_k$ are optimized. The specific RBF terrain model is introduced in Sec. IV, while the per-wheel contact residual and the complete scan-matching objective are derived in Sec. V.

IV. THE RBF-BASED TERRAIN APPROXIMATION

Based on the formulation in Sec. III, this section introduces the RBF-based terrain approximation method. In particular, we first introduce the RBF approximation model for the terrain and then present the recursive ridge regression used for the weight update. Note that, in this section, the preliminary base pose used to express the LiDAR terrain observations in the world frame is treated as given; its construction is described in Sec. V-B.

A. The RBF Approximation Model

The RBF approximation is universal [17], i.e., given enough centers, it can approximate any continuous function on a compact set to arbitrary accuracy. Consequently, the RBF approximation model gives a dense, smooth representation for the terrain. Compared to 2.5-D elevation maps, the RBF approximation model can predict the height at any arbitrary point within the compact Region of Interest (ROI) $\mathcal{X}$. Moreover, its spatial gradient can be calculated explicitly at any point, which is essential for solving the optimization problem for the LIO.

To this end, we let the RBF approximation for the terrain be

$$f(\mathbf{x}; \mathbf{w}) = \sum_{i=1}^N w_i \phi(\|\mathbf{x} - \mathbf{c}_i\|_2), \quad (8)$$

where

- $\mathbf{x} = [p_x, p_y]^\top \in \mathbb{R}^2$ is a 2D query point in the horizontal plane, and $f(\mathbf{x}; \mathbf{w})$ denotes the predicted terrain height at the query point;
- $\mathbf{c}_{1:N} := \{\mathbf{c}_i = (x_i^c, y_i^c) \in \mathcal{X}\}_{i=1}^N \subset \mathbb{R}^2$ are the RBF center points, which will be selected adaptively based on the distribution of the terrain point cloud;
- $\mathbf{w} := \{w_i \in \mathbb{R}\}_{i=1}^N$ are the weights of the RBFs;
- $\phi(\cdot)$ is the RBF. In this work, we use the Gaussian kernel, namely,

$$\phi(r_i) = e^{-\frac{(r_i)^2}{2\sigma^2}} := \kappa_\sigma(\mathbf{x}; \mathbf{c}_i), \quad (9)$$

where $r_i = \|\mathbf{x} - \mathbf{c}_i\|_2$ is the Euclidean distance between the query point $\mathbf{x}$ and center $\mathbf{c}_i$, and $\sigma \in \mathbb{R}$ is a bandwidth parameter that determines the spatial influence of the kernel.

Notably, the model (8) predicts the terrain height as a continuous function of 2D spatial coordinates. The number and placement of the RBF centers critically affect the approximation quality. In general, more centers improve the representation capability of the model. However, since the terrain is reconstructed from only a finite set of LiDAR measurements, using too many centers increases the computational cost and may also make the weight estimation numerically unstable. Moreover, due to the rapid decay of the Gaussian kernel, a center that is not sufficiently supported by nearby LiDAR points contributes little useful information to the regression. Hence, the number and placement of the centers must be carefully chosen to balance approximation accuracy, numerical stability, and computational efficiency.

To this end, we use an adaptive strategy to select the RBF centers, as illustrated in Fig. 1. First, we build a k-d

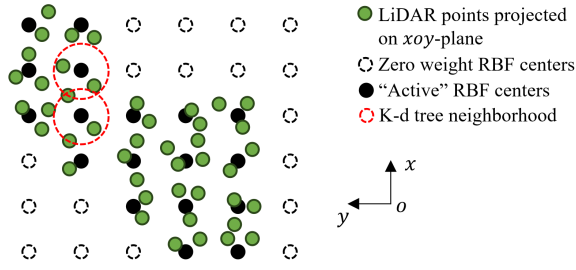


Fig. 1. The RBF center selection strategy illustration on xoy -plane. Only candidate centers with sufficient nearby LiDAR support are retained.

tree from the terrain point cloud for efficient neighborhood queries. Next, candidate centers are generated on a mesh with a specified resolution over the compact ROI $\mathcal{X}$ in the xoy -plane. For each candidate center, the k-d tree counts the number of LiDAR points within a neighbourhood of radius r_{kd} . The candidate center is accepted only if this number exceeds a pre-defined threshold. In our implementation, we set $r_{kd} = \sigma$ so that the neighborhood test is consistent with the effective spatial support of the Gaussian kernel. Empirically, a minimum of three nearby LiDAR points is sufficient to yield reliable center selection. Such a strategy balances approximation accuracy, numerical stability, and computational efficiency.

B. The RBF Weight Estimate by Ridge Regression

To use (8) to approximate the terrain, we need to estimate the RBF weights. When no prior information about the terrain is available, e.g., at initialization, we estimate the RBF weights by solving a ridge regression problem. More specifically, since the robot's base pose is assumed to be known in this section, let $\{\mathbf{p}_k^i \in \mathbb{R}^3\}_{i=1}^{M_k}$, ($i = 1, \dots, M_k$), denote the LiDAR point cloud data at the k -th time step expressed in the world frame, where the points $\mathbf{p}_k^i := [\mathbf{x}_k^{i\top}, p_{z,k}^i]^\top$ are approximately i.i.d. samples of the stochastic vector $\mathbf{p}$, and $\mathbf{x}_k^i := [p_{x,k}^i, p_{y,k}^i]^\top \in \mathbb{R}^2$. In addition, for the purpose of estimation, we model the observed LiDAR points $\mathbf{p}_k^i$, $\forall i, k$, expressed in the world frame, as samples of a stochastic vector $\mathbf{p}$. More specifically, we assume that

$$\mathbf{p} := \begin{bmatrix} \mathbf{x} \\ p_z \end{bmatrix} = \bar{\mathbf{p}} + \boldsymbol{\epsilon} = \begin{bmatrix} \bar{\mathbf{x}} \\ \bar{p}_z \end{bmatrix} + \begin{bmatrix} \boldsymbol{\epsilon}_x \\ \epsilon_z \end{bmatrix}, \quad (10)$$

where $\bar{\mathbf{p}}$ is a stochastic vector of 3D coordinates randomly drawn from the terrain surface expressed in the world frame, $\bar{\mathbf{x}} = [p_x, p_y]^\top$, and $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \sigma_\epsilon^2 I)$ denotes the point cloud measurement noise, which is assumed to be independent of $\bar{\mathbf{p}}$. Further, suppose that (8) approximates the terrain surface well on $\mathcal{X}$, i.e.,

$$p_z = \underbrace{[\kappa_\sigma(\bar{\mathbf{x}}; \mathbf{c}_1) \quad \cdots \quad \kappa_\sigma(\bar{\mathbf{x}}; \mathbf{c}_N)]}_{\boldsymbol{\Phi}(\bar{\mathbf{x}})^\top} \underbrace{\begin{bmatrix} w_1 \\ \vdots \\ w_N \end{bmatrix}}_{\mathbf{w}} + \epsilon_z, \quad (11)$$

where the centers $\mathbf{c}_j \in \mathcal{X}$, $j = 1, \dots, N$, and $\mathbf{w}$ is the vector of RBF weights.

Next, recall that our goal is to fit the terrain surface by estimating the RBF weights $\mathbf{w}$. However, we do not have access to i.i.d. samples of $\bar{\mathbf{x}}$, but only to the noisy observations $\mathbf{x}_k^i$ of $\mathbf{x}$. To this end, let $\mathbf{m}(\mathbf{x}) := \mathbb{E}[\boldsymbol{\Phi}(\bar{\mathbf{x}}) | \mathbf{x}]$. Since ϵ_z is independent of $\mathbf{x}$, it holds that

$$\mathbb{E}[p_z | \mathbf{x}] = \mathbb{E}[\boldsymbol{\Phi}(\bar{\mathbf{x}})^\top \mathbf{w} + \epsilon_z | \mathbf{x}] = \mathbb{E}[\boldsymbol{\Phi}(\bar{\mathbf{x}})^\top | \mathbf{x}] \mathbf{w} = \mathbf{m}(\mathbf{x})^\top \mathbf{w}. \quad (12)$$

Therefore, by the tower property of conditional expectation, it follows that

$$\begin{aligned} \mathbb{E}[\mathbf{m}(\mathbf{x}) p_z] &= \mathbb{E}[\mathbb{E}[\mathbf{m}(\mathbf{x}) p_z | \mathbf{x}]] = \mathbb{E}[\mathbf{m}(\mathbf{x}) \mathbb{E}[p_z | \mathbf{x}]] \\ &= \mathbb{E}[\mathbf{m}(\mathbf{x}) \mathbf{m}(\mathbf{x})^\top] \mathbf{w}. \end{aligned} \quad (13)$$

Notably, (13) forms a moment equality, from which we obtain the estimator at the k -th time step:

$$\begin{aligned} \hat{\mathbf{w}}_k &= \mathbb{E}[\mathbf{m}(\mathbf{x}) \mathbf{m}(\mathbf{x})^\top]^{-1} \mathbb{E}[\mathbf{m}(\mathbf{x}) p_z] \\ &\approx \underbrace{\left(\sum_{t=1}^k \sum_{i=1}^{M_t} \mathbf{m}(\mathbf{x}_t^i) \mathbf{m}(\mathbf{x}_t^i)^\top \right)^{-1}}_{\mathbf{H}_k} \underbrace{\left(\sum_{t=1}^k \sum_{i=1}^{M_t} \mathbf{m}(\mathbf{x}_t^i) p_{z,t}^i \right)}_{\mathbf{b}_k}, \end{aligned} \quad (14)$$

where

$$\begin{aligned} \mathbf{m}(\mathbf{x}_t^i) &= \mathbb{E}[\boldsymbol{\Phi}(\bar{\mathbf{x}}) | \mathbf{x}_t^i] \\ &= \frac{\sigma^2}{\tilde{\sigma}^2} [\kappa_{\tilde{\sigma}}(\mathbf{x}_t^i; \mathbf{c}_1) \quad \cdots \quad \kappa_{\tilde{\sigma}}(\mathbf{x}_t^i; \mathbf{c}_N)]^\top, \end{aligned} \quad (15)$$

with $\tilde{\sigma} = \sqrt{\sigma^2 + \sigma_\epsilon^2}$, since $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \sigma_\epsilon^2 I)$. Moreover, (14) is the unique minimizer of

$$\min_{\mathbf{w}} \sum_{t=1}^k \sum_{i=1}^{M_t} (p_{z,t}^i - \mathbf{m}(\mathbf{x}_t^i)^\top \mathbf{w})^2. \quad (16)$$

However, $\mathbf{H}_k$ may be ill-conditioned due to the locality and overlap structure of the RBF kernels, which can significantly affect the approximation accuracy. To address this issue, we add a regularization term to (16) and solve a ridge regression problem [18], i.e.,

$$\min_{\mathbf{w}} \sum_{t=1}^k \sum_{i=1}^{M_t} (p_{z,t}^i - \mathbf{m}(\mathbf{x}_t^i)^\top \mathbf{w})^2 + \lambda \|\mathbf{w}\|^2, \quad (17)$$

and the corresponding estimator is

$$\hat{\mathbf{w}}_k^{RG} = \underbrace{(\mathbf{H}_k + \lambda I)}_{\mathcal{H}_k}^{-1} \mathbf{b}_k. \quad (18)$$

In particular, the ridge penalty $\lambda \|\mathbf{w}\|^2$ mitigates the ill-conditioning of $\mathbf{H}_k$ and numerically stabilizes the least-squares estimate in (16). Although this regularization introduces bias, the bias is typically modest in practice, while the reduction in variance may lead to a smaller mean-squared error [18]. The choice of λ will be further discussed in Sec. VII.

C. The Recursive RBF Weight Update

Recall the terrain approximation model (8). Since the RBF (9) decays exponentially to zero as r_i increases, the kernel $\kappa_\sigma(\mathbf{x}; \mathbf{c}_i)$ only captures the terrain geometry in the vicinity of the center $\mathbf{c}_i \in \mathcal{X}$. Therefore, at a new time step $k+1$, only those RBF centers that are sufficiently supported by the newly

observed LiDAR points contribute significantly to the update. We refer to such centers as the *active RBF centers*, and the remaining ones as the *inactive RBF centers*.

According to (14), the updated estimator from the newly received LiDAR point cloud at time step $k+1$ takes the form

$$\hat{\mathbf{w}}_{k+1}^{RG} = (\mathcal{H}_{k+1})^{-1}(\mathbf{b}_k + \mathbf{m}_{k+1}\mathbf{p}_{z,k+1}), \quad (19a)$$

$$\mathcal{H}_{k+1} = \mathcal{H}_k + \mathbf{m}_{k+1}\mathbf{m}_{k+1}^\top, \quad (19b)$$

where $\mathbf{m}_{k+1} \in \mathbb{R}^{N \times M_{k+1}}$ and $\mathbf{p}_{z,k+1} \in \mathbb{R}^{M_{k+1}}$. More specifically,

$$\begin{aligned} \mathbf{m}_{k+1} &:= \begin{bmatrix} \mathbf{m}(\mathbf{x}_{k+1}^1) & \dots & \mathbf{m}(\mathbf{x}_{k+1}^{M_{k+1}}) \end{bmatrix} \\ &= \frac{\sigma^2}{\tilde{\sigma}^2} \begin{bmatrix} \kappa_{\tilde{\sigma}}(\mathbf{x}_{k+1}^1; \mathbf{c}_1) & \dots & \kappa_{\tilde{\sigma}}(\mathbf{x}_{k+1}^{M_{k+1}}; \mathbf{c}_1) \\ \vdots & \ddots & \vdots \\ \kappa_{\tilde{\sigma}}(\mathbf{x}_{k+1}^1; \mathbf{c}_N) & \dots & \kappa_{\tilde{\sigma}}(\mathbf{x}_{k+1}^{M_{k+1}}; \mathbf{c}_N) \end{bmatrix}, \end{aligned} \quad (20a)$$

$$\approx \begin{bmatrix} \mathbf{0} \\ \tilde{\mathbf{m}}_{k+1} \\ \mathbf{0} \end{bmatrix}, \quad (20b)$$

$$\mathbf{p}_{z,k+1} := \begin{bmatrix} p_{z,k+1}^1 & \dots & p_{z,k+1}^{M_{k+1}} \end{bmatrix}^\top. \quad (20c)$$

Here, $\tilde{\mathbf{m}}_{k+1}$ collects the scaled active rows of $\mathbf{m}_{k+1}$, including the factor $\sigma^2/\tilde{\sigma}^2$. Since each LiDAR scan covers only a local portion of $\mathcal{X}$ and $\kappa_{\tilde{\sigma}}(\mathbf{x}; \mathbf{c}_j)$ decays rapidly with distance, the remaining rows are numerically close to zero. For notational convenience, (20b) groups the active rows into a contiguous block. Consequently, the increment in (19b) is confined to the active sub-block, giving $\mathcal{H}_k$ a localized sparse structure. Since $\mathbf{b}_k = \mathcal{H}_k \hat{\mathbf{w}}_k^{RG}$, (19a) can be rewritten in a Kalman-filter-style form as

$$\begin{aligned} \hat{\mathbf{w}}_{k+1}^{RG} &= \mathcal{H}_{k+1}^{-1} \mathbf{b}_k + \mathcal{H}_{k+1}^{-1} \mathbf{m}_{k+1} \mathbf{p}_{z,k+1} \\ &= \mathcal{H}_{k+1}^{-1} \mathcal{H}_k \hat{\mathbf{w}}_k^{RG} + \mathcal{H}_{k+1}^{-1} \mathbf{m}_{k+1} \mathbf{p}_{z,k+1} \\ &= \mathcal{H}_{k+1}^{-1} (\mathcal{H}_{k+1} - \mathbf{m}_{k+1} \mathbf{m}_{k+1}^\top) \hat{\mathbf{w}}_k^{RG} + \mathcal{H}_{k+1}^{-1} \mathbf{m}_{k+1} \mathbf{p}_{z,k+1} \\ &= \hat{\mathbf{w}}_k^{RG} + \mathcal{H}_{k+1}^{-1} \mathbf{m}_{k+1} [\mathbf{p}_{z,k+1} - \mathbf{m}_{k+1}^\top \hat{\mathbf{w}}_k^{RG}]. \end{aligned} \quad (21)$$

Because a LiDAR point has negligible kernel values at RBF centers far from it, the off-diagonal blocks of $\mathcal{H}_{k+1}$ between the current active centers and distant inactive centers are numerically small. Neglecting these blocks gives a block-diagonal approximation of $\mathcal{H}_{k+1}$, while ridge regularization stabilizes the inversion of the retained diagonal blocks. We partition $\hat{\mathbf{w}}_{k+1}^{RG}$ conformably and approximate (21) as

$$\begin{aligned} \begin{bmatrix} \hat{\mathbf{w}}_{1,k+1}^{RG} \\ \hat{\mathbf{w}}_{2,k+1}^{RG} \\ \hat{\mathbf{w}}_{3,k+1}^{RG} \end{bmatrix} &\approx \begin{bmatrix} \hat{\mathbf{w}}_{1,k}^{RG} \\ \hat{\mathbf{w}}_{2,k}^{RG} \\ \hat{\mathbf{w}}_{3,k}^{RG} \end{bmatrix} + \begin{bmatrix} \mathcal{H}_{1,k+1}^{-1} & & \\ & \mathcal{H}_{2,k+1}^{-1} & \\ & & \mathcal{H}_{3,k+1}^{-1} \end{bmatrix} \begin{bmatrix} \mathbf{0} \\ \tilde{\mathbf{m}}_{k+1} \\ \mathbf{0} \end{bmatrix} \\ &\times \begin{bmatrix} \mathbf{p}_{z,k+1} - [\mathbf{0}^\top & \tilde{\mathbf{m}}_{k+1}^\top & \mathbf{0}^\top] \begin{bmatrix} \hat{\mathbf{w}}_{1,k}^{RG} \\ \hat{\mathbf{w}}_{2,k}^{RG} \\ \hat{\mathbf{w}}_{3,k}^{RG} \end{bmatrix} \end{bmatrix} \\ &= \begin{bmatrix} \hat{\mathbf{w}}_{2,k}^{RG} + \mathcal{H}_{2,k+1}^{-1} \tilde{\mathbf{m}}_{k+1} (\mathbf{p}_{z,k+1} - \tilde{\mathbf{m}}_{k+1}^\top \hat{\mathbf{w}}_{2,k}^{RG}) \\ \hat{\mathbf{w}}_{3,k}^{RG} \end{bmatrix}. \end{aligned} \quad (22)$$

Under the same approximation, the Woodbury matrix identity gives

$$\begin{aligned} \mathcal{H}_{k+1}^{-1} &= \mathcal{H}_k^{-1} - \mathcal{H}_k^{-1} \mathbf{m}_{k+1} (I + \mathbf{m}_{k+1}^\top \mathcal{H}_k^{-1} \mathbf{m}_{k+1})^{-1} \mathbf{m}_{k+1}^\top \mathcal{H}_k^{-1} \\ &\approx \begin{bmatrix} \mathcal{H}_{1,k}^{-1} & & \\ & \mathcal{H}_{2,k+1}^{-1} & \\ & & \mathcal{H}_{3,k}^{-1} \end{bmatrix}, \end{aligned} \quad (23)$$

where

$$\mathcal{H}_{2,k+1}^{-1} := \mathcal{H}_{2,k}^{-1} - \mathcal{H}_{2,k}^{-1} \tilde{\mathbf{m}}_{k+1} (I + \tilde{\mathbf{m}}_{k+1}^\top \mathcal{H}_{2,k}^{-1} \tilde{\mathbf{m}}_{k+1})^{-1} \tilde{\mathbf{m}}_{k+1}^\top \mathcal{H}_{2,k}^{-1}. \quad (24)$$

Equations (22) and (23) update only the active weights and keep the inactive weights fixed, as illustrated in Fig. 2. Applying the recursion only to the active block substantially reduces computation by exploiting the sparse local structure.

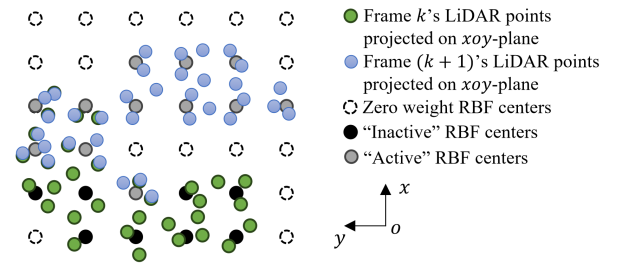


Fig. 2. Illustration of the recursive RBF weight update on the xoy -plane. Only the weights associated with the active RBF centers (gray) are updated, since they are supported by the newly received observations.

V. THE LIO WITH THE MANIFOLD CONSTRAINT

The previous section provides an approximation of the terrain. We now use this approximation as a “soft constraint” in the LIO system to improve the localization accuracy of a legged-wheel robot maneuvering on the terrain. This constraint introduces additional geometric regularization when estimating the robot base pose, particularly in the vertical direction.

A. Leg Kinematics and Wheel–Terrain Contact

To exploit the manifold constraints, we first characterize the wheel–terrain contact geometry. To this end, let $\mathbf{q}_k$ denote the joint configuration at the time step k . For each wheel $j \in \mathcal{W}$, we can easily calculate the wheel center by the forward kinematics of the leg $\mathbf{h}_j(\mathbf{q})$, and hence the coordinate of wheel center ξ_j expressed in the world frame takes the form

$$\xi_j({}^0\mathbf{R}_B, {}^0\mathbf{t}_B; \mathbf{q}) := \begin{bmatrix} \xi_{x,j} \\ \xi_{y,j} \\ \xi_{z,j} \end{bmatrix} = {}^0\mathbf{R}_B \mathbf{h}_j(\mathbf{q}) + {}^0\mathbf{t}_B, \quad (25)$$

where ${}^0\mathbf{R}_B \in \text{SO}(3)$ and ${}^0\mathbf{t}_B \in \mathbb{R}^3$ denote the rotation matrix and translation vector that transform from the base frame to the world frame. Similarly, the unit vector along the wheel axis $\mathbf{u}_j$ in the world frame is given by

$$\mathbf{u}_j({}^0\mathbf{R}_B; \mathbf{q}) = {}^0\mathbf{R}_B \mathbf{g}_j(\mathbf{q}), \quad (26)$$

with $\mathbf{g}_j(\mathbf{q})$ being the axis directions in the base frame derived from leg kinematics.

Moreover, we assume that each wheel admits a unique effective contact point on the terrain at each time step. Let r be the wheel radius and $\chi_j = [\chi_{j,x}, \chi_{j,y}, \chi_{j,z}]^\top \in \mathbb{R}^3$ denote the contact point between the wheel and the terrain. This point must satisfy the following geometric contact constraints:

$$\text{On terrain: } \chi_{j,z} - f(\chi_{j,x}, \chi_{j,y}) = 0, \quad (27a)$$

$$\text{In wheel plane: } \mathbf{u}_j({}^0\mathbf{R}_B; \mathbf{q})^\top \mathbf{n}_\omega(\chi_j, {}^0\mathbf{R}_B, {}^0\mathbf{t}_B; \mathbf{q}) = 0, \quad (27b)$$

$$\text{Radius condition: } \|\chi_j - \xi_j({}^0\mathbf{R}_B, {}^0\mathbf{t}_B; \mathbf{q})\|^2 - r^2 = 0, \quad (27c)$$

$$\text{Normal collinearity: } \mathbf{n}_\omega(\chi_j, {}^0\mathbf{R}_B, {}^0\mathbf{t}_B; \mathbf{q}) \times \mathbf{n}_g(\chi_j) = \mathbf{0}, \quad (27d)$$

where f is the RBF-based terrain height function defined in (8), $\mathbf{n}_g(\chi_j)$ is the terrain normal at the contact point computed from the gradient of f , $\mathbf{n}_\omega(\chi_j, {}^0\mathbf{R}_B, {}^0\mathbf{t}_B; \mathbf{q})$ is the radial unit vector from the wheel center to the contact point. Specifically,

$$\mathbf{n}_\omega(\chi_j, {}^0\mathbf{R}_B, {}^0\mathbf{t}_B; \mathbf{q}) := \frac{\chi_j - \xi_j({}^0\mathbf{R}_B, {}^0\mathbf{t}_B; \mathbf{q})}{r}, \quad (28a)$$

$$\mathbf{n}_g(\chi_j) := \frac{1}{\sqrt{1 + \left(\frac{\partial f}{\partial p_x}\right)^2 + \left(\frac{\partial f}{\partial p_y}\right)^2}} \begin{bmatrix} -\frac{\partial f}{\partial p_x}(\chi_{j,x}, \chi_{j,y}) \\ -\frac{\partial f}{\partial p_y}(\chi_{j,x}, \chi_{j,y}) \\ 1 \end{bmatrix}. \quad (28b)$$

We use (28) as geometric consistency conditions in the soft-constrained optimization framework described below.

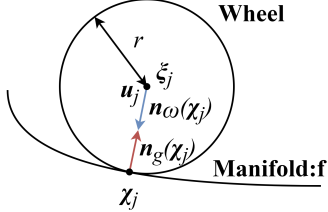


Fig. 3. Illustration of wheel-terrain contact.

B. IMU Pre-integration

Due to the relatively low frame rate of LiDAR scans, direct scan matching without a reliable initial guess is prone to convergence issues, especially under abrupt motions. To mitigate this issue, we use an IMU pre-integration module to integrate the high-rate inertial measurements between two consecutive LiDAR frames and provide **inertial motion information** for scan matching.

To this end, let the **inertial propagation** state at time step k be

$$\mathbf{s}_k = ({}^0\mathbf{R}_{B,k}, {}^0\mathbf{t}_{B,k}, {}^0\mathbf{v}_{B,k}, \mathbf{b}_k^a, \mathbf{b}_k^g), \quad (29)$$

where ${}^0\mathbf{R}_{B,k}$ and ${}^0\mathbf{t}_{B,k}$ denote the **pose variables introduced in (1)**, ${}^0\mathbf{v}_{B,k} \in \mathbb{R}^3$ is the linear velocity expressed in the world frame, and $\mathbf{b}_k^a, \mathbf{b}_k^g \in \mathbb{R}^3$ are the accelerometer and gyroscope biases expressed in the local IMU frame, respectively.

Given the IMU measurements ${}^B\mathbf{a}_{m,k} \in \mathbb{R}^3$ and ${}^B\boldsymbol{\omega}_{m,k} \in \mathbb{R}^3$, where ${}^B\mathbf{a}_{m,k}$ and ${}^B\boldsymbol{\omega}_{m,k}$ denote the specific force and

angular velocity expressed in the IMU frame, respectively, the state is propagated recursively with sampling interval Δt as

$${}^0\mathbf{R}_{B,k+1} = {}^0\mathbf{R}_{B,k} \exp([\mathbf{B}\boldsymbol{\omega}_{m,k} - \mathbf{b}_k^g]_\times \Delta t), \quad (30a)$$

$${}^0\mathbf{v}_{B,k+1} = {}^0\mathbf{v}_{B,k} + [{}^0\mathbf{g} + {}^0\mathbf{R}_{B,k}({}^B\mathbf{a}_{m,k} - \mathbf{b}_k^a)] \Delta t, \quad (30b)$$

$${}^0\mathbf{t}_{B,k+1} = {}^0\mathbf{t}_{B,k} + {}^0\mathbf{v}_{B,k} \Delta t + \frac{1}{2} [{}^0\mathbf{g} + {}^0\mathbf{R}_{B,k}({}^B\mathbf{a}_{m,k} - \mathbf{b}_k^a)] \Delta t^2. \quad (30c)$$

Here, ${}^0\mathbf{g}$ denotes the gravity vector in the world frame, $\exp(\cdot)$ denotes the matrix exponential on the Lie algebra $\mathfrak{so}(3)$, and $[\cdot]_\times$ maps a vector in $\mathbb{R}^3$ to its corresponding skew-symmetric matrix in $\mathfrak{so}(3)$. The pre-integrated quantities over the interval $[t_k, t_{k+1}]$ are supplied to the scan-matching stage in the LIO framework.

C. The Manifold Constraints

We assume that the wheels maintain firm contact with the terrain. Accordingly, for each wheel $j \in \mathcal{W}$, we define a manifold residual vector by stacking the three scalar contact constraints together with the vector-valued normal-collinearity residual:

$$\mathbf{r}_{\mathcal{M}}^j(\boldsymbol{\eta}_k, \chi_j) := \begin{bmatrix} r_{j,1} \\ r_{j,2} \\ r_{j,3} \\ r_{j,4} \end{bmatrix} = \begin{bmatrix} \chi_{j,z} - f(\chi_{j,x}, \chi_{j,y}) \\ \mathbf{u}_j({}^0\mathbf{R}_B; \mathbf{q})^\top \mathbf{n}_\omega(\chi_j, {}^0\mathbf{R}_B, {}^0\mathbf{t}_B; \mathbf{q}) \\ \|\chi_j - \xi_j({}^0\mathbf{R}_B, {}^0\mathbf{t}_B; \mathbf{q})\|^2 - r^2 \\ \mathbf{n}_\omega(\chi_j, {}^0\mathbf{R}_B, {}^0\mathbf{t}_B; \mathbf{q}) \times \mathbf{n}_g(\chi_j) \end{bmatrix} \approx \mathbf{0}. \quad (31)$$

Next, we incorporate the manifold residuals into the feature-based scan-matching objective. Following the LIO-SAM frontend [1], the IMU pre-integration module supplies inertial motion information for each LiDAR frame, while point-to-line and point-to-plane residuals form the original scan-matching cost:

$$J_{\text{scan}}(\boldsymbol{\eta}_k) = \sum_{\mathbf{p}_{s,k}^e \in \mathcal{F}_k^e} \|\mathbf{d}_e({}^0\mathbf{R}_{B,k}, {}^0\mathbf{t}_{B,k}; \mathbf{p}_{s,k}^e)\|^2 + \sum_{\mathbf{p}_{s,k}^p \in \mathcal{F}_k^p} \|\mathbf{d}_p({}^0\mathbf{R}_{B,k}, {}^0\mathbf{t}_{B,k}; \mathbf{p}_{s,k}^p)\|^2, \quad (32)$$

where $\mathcal{F}_k^e$ and $\mathcal{F}_k^p$ denote the edge and planar feature sets, and $\mathbf{d}_e(\cdot)$ and $\mathbf{d}_p(\cdot)$ are the corresponding point-to-line and point-to-plane residuals.

We then incorporate the manifold residuals into the scan-matching objective as soft constraints for pose regularization. The corresponding manifold cost is defined as

$$J_{\mathcal{M}}(\boldsymbol{\eta}_k, \boldsymbol{\chi}_k) = \sum_{j \in \mathcal{W}} \|\mathbf{r}_{\mathcal{M}}^j(\boldsymbol{\eta}_k, \chi_j)\|^2, \quad (33)$$

where **all contact points in $\boldsymbol{\chi}_k$** are jointly optimized with the pose variables $\boldsymbol{\eta}_k$ in the manifold-constraint term. **Accordingly**, the **manifold-augmented scan-matching** cost takes the form

$$J_{\text{aug}}(\boldsymbol{\eta}_k, \boldsymbol{\chi}_k) = J_{\text{scan}}(\boldsymbol{\eta}_k) + J_{\mathcal{M}}(\boldsymbol{\eta}_k, \boldsymbol{\chi}_k). \quad (34)$$

In particular, we minimize (34) using the Levenberg-Marquardt algorithm [19]. Since the Jacobians of the point-to-line and point-to-plane residuals have been discussed in

detail in [1], we only derive the Jacobian of the newly introduced manifold residual $\mathbf{r}_{\mathcal{M}}^j(\boldsymbol{\eta}_k, \boldsymbol{\chi}_j)$ here. In the Levenberg–Marquardt solver, this residual is linearized around the current estimate with respect to the local pose increment and the contact-point increment. Accordingly, for each wheel, we define the joint increment vector as

$$\delta \mathbf{x}_j = \begin{bmatrix} \delta \boldsymbol{\eta} \\ \delta \boldsymbol{\chi}_j \end{bmatrix}, \quad \delta \boldsymbol{\eta} = \begin{bmatrix} \delta \boldsymbol{\theta} \\ \delta \mathbf{t} \end{bmatrix},$$

where $\delta \boldsymbol{\eta}$ denotes the local increment of the base pose. The corresponding Jacobian takes the block form

$$\mathbf{J}_j = \frac{\partial \mathbf{r}_{\mathcal{M}}^j}{\partial \delta \mathbf{x}_j} = \begin{bmatrix} \frac{\partial \mathbf{r}_{\mathcal{M}}^j}{\partial \delta \boldsymbol{\theta}} & \frac{\partial \mathbf{r}_{\mathcal{M}}^j}{\partial \delta \mathbf{t}} & \frac{\partial \mathbf{r}_{\mathcal{M}}^j}{\partial \delta \boldsymbol{\chi}_j} \end{bmatrix}. \quad (35)$$

Using a right-perturbation model for the rotation increment $\delta \boldsymbol{\theta}$, the derivatives of the wheel center and wheel-axis direction with respect to the pose increments are

$$\begin{aligned} \frac{\partial \boldsymbol{\xi}_j}{\partial \delta \boldsymbol{\theta}} &= -{}^{\mathbf{O}}\mathbf{R}_B [\mathbf{h}_j(\mathbf{q})]_{\times}, & \frac{\partial \boldsymbol{\xi}_j}{\partial \delta \mathbf{t}} &= \mathbf{I}_3, \\ \frac{\partial \mathbf{u}_j}{\partial \delta \boldsymbol{\theta}} &= -[\mathbf{u}_j]_{\times} {}^{\mathbf{O}}\mathbf{R}_B, & \frac{\partial \mathbf{u}_j}{\partial \delta \mathbf{t}} &= \mathbf{0}. \end{aligned} \quad (36)$$

With these preliminaries, we derive the Jacobian blocks for each residual component in (31). Let $\mathbf{d}_j := \boldsymbol{\chi}_j - \boldsymbol{\xi}_j$. It follows that

- **On terrain:** $r_{j,1} = \chi_{j,z} - f(\chi_{j,x}, \chi_{j,y})$. Its Jacobian takes the form

$$\frac{\partial r_{j,1}}{\partial \delta \boldsymbol{\theta}} = \mathbf{0}_{1 \times 3}, \quad \frac{\partial r_{j,1}}{\partial \delta \mathbf{t}} = \mathbf{0}_{1 \times 3}, \quad \frac{\partial r_{j,1}}{\partial \delta \boldsymbol{\chi}_j} = \begin{bmatrix} -\frac{\partial f}{\partial x} & -\frac{\partial f}{\partial y} & 1 \end{bmatrix}. \quad (37)$$

- **In wheel plane:** $r_{j,2} = \mathbf{u}_j({}^{\mathbf{O}}\mathbf{R}_B; \mathbf{q})^{\top} \mathbf{n}_{\omega}(\boldsymbol{\chi}_j, {}^{\mathbf{O}}\mathbf{R}_B, {}^{\mathbf{O}}\mathbf{t}_B; \mathbf{q})$. Since $\mathbf{n}_{\omega} = \mathbf{d}_j/r$, it follows that

$$\begin{aligned} \frac{\partial r_{j,2}}{\partial \delta \boldsymbol{\theta}} &= \frac{1}{r} (\mathbf{u}_j^{\top} {}^{\mathbf{O}}\mathbf{R}_B [\mathbf{h}_j(\mathbf{q})]_{\times} - \mathbf{d}_j^{\top} [\mathbf{u}_j]_{\times} {}^{\mathbf{O}}\mathbf{R}_B), \\ \frac{\partial r_{j,2}}{\partial \delta \mathbf{t}} &= -\frac{\mathbf{u}_j^{\top}}{r}, & \frac{\partial r_{j,2}}{\partial \delta \boldsymbol{\chi}_j} &= \frac{\mathbf{u}_j^{\top}}{r}. \end{aligned} \quad (38)$$

- **Radius condition:** $r_{j,3} = \|\boldsymbol{\chi}_j - \boldsymbol{\xi}_j({}^{\mathbf{O}}\mathbf{R}_B, {}^{\mathbf{O}}\mathbf{t}_B; \mathbf{q})\|^2 - r^2$. Its Jacobian takes the form

$$\begin{aligned} \frac{\partial r_{j,3}}{\partial \delta \boldsymbol{\theta}} &= 2 \mathbf{d}_j^{\top} {}^{\mathbf{O}}\mathbf{R}_B [\mathbf{h}_j(\mathbf{q})]_{\times}, \\ \frac{\partial r_{j,3}}{\partial \delta \mathbf{t}} &= -2 \mathbf{d}_j^{\top}, & \frac{\partial r_{j,3}}{\partial \delta \boldsymbol{\chi}_j} &= 2 \mathbf{d}_j^{\top}. \end{aligned} \quad (39)$$

- **Normal collinearity:** $\mathbf{r}_{j,4} = \mathbf{n}_{\omega}(\boldsymbol{\chi}_j, {}^{\mathbf{O}}\mathbf{R}_B, {}^{\mathbf{O}}\mathbf{t}_B; \mathbf{q}) \times \mathbf{n}_g(\boldsymbol{\chi}_j)$. First, the derivatives of $\mathbf{n}_{\omega}$ take the form

$$\frac{\partial \mathbf{n}_{\omega}}{\partial \delta \boldsymbol{\theta}} = \frac{1}{r} {}^{\mathbf{O}}\mathbf{R}_B [\mathbf{h}_j(\mathbf{q})]_{\times}, \quad \frac{\partial \mathbf{n}_{\omega}}{\partial \delta \mathbf{t}} = -\frac{1}{r} \mathbf{I}_3, \quad \frac{\partial \mathbf{n}_{\omega}}{\partial \delta \boldsymbol{\chi}_j} = \frac{1}{r} \mathbf{I}_3. \quad (40)$$

On the other hand, let

$$\mathbf{g}_f := \begin{bmatrix} -\frac{\partial f}{\partial p_x}(\chi_{j,x}, \chi_{j,y}) \\ -\frac{\partial f}{\partial p_y}(\chi_{j,x}, \chi_{j,y}) \\ 1 \end{bmatrix}, \quad \mathbf{H}_f := \begin{bmatrix} -\frac{\partial^2 f}{\partial p_x^2} & -\frac{\partial^2 f}{\partial p_x \partial p_y} & 0 \\ -\frac{\partial^2 f}{\partial p_y \partial p_x} & -\frac{\partial^2 f}{\partial p_y^2} & 0 \\ 0 & 0 & 0 \end{bmatrix}. \quad (41)$$

Then it holds that $\frac{\partial \mathbf{n}_g}{\partial \delta \boldsymbol{\chi}_j} = \frac{1}{\|\mathbf{g}_f\|_2} (\mathbf{I}_3 - \mathbf{n}_g \mathbf{n}_g^{\top}) \mathbf{H}_f$. Applying the product rule for cross products yields

$$\begin{aligned} \frac{\partial \mathbf{r}_{j,4}}{\partial \delta \boldsymbol{\theta}} &= -[\mathbf{n}_g]_{\times} \frac{\partial \mathbf{n}_{\omega}}{\partial \delta \boldsymbol{\theta}} = -\frac{1}{r} [\mathbf{n}_g]_{\times} {}^{\mathbf{O}}\mathbf{R}_B [\mathbf{h}_j(\mathbf{q})]_{\times}, \\ \frac{\partial \mathbf{r}_{j,4}}{\partial \delta \mathbf{t}} &= -[\mathbf{n}_g]_{\times} \frac{\partial \mathbf{n}_{\omega}}{\partial \delta \mathbf{t}} = \frac{1}{r} [\mathbf{n}_g]_{\times}, \\ \frac{\partial \mathbf{r}_{j,4}}{\partial \delta \boldsymbol{\chi}_j} &= -[\mathbf{n}_g]_{\times} \frac{\partial \mathbf{n}_{\omega}}{\partial \delta \boldsymbol{\chi}_j} + [\mathbf{n}_{\omega}]_{\times} \frac{\partial \mathbf{n}_g}{\partial \delta \boldsymbol{\chi}_j} \\ &= -\frac{1}{r} [\mathbf{n}_g]_{\times} + \frac{1}{\|\mathbf{g}_f\|_2} [\mathbf{n}_{\omega}]_{\times} (\mathbf{I}_3 - \mathbf{n}_g \mathbf{n}_g^{\top}) \mathbf{H}_f. \end{aligned} \quad (42)$$

All these Jacobian blocks are assembled into the full matrix $\mathbf{J}_j$ in (35). The overall Jacobian of the manifold cost $J_{\mathcal{M}}$ is then obtained by stacking the contributions from **all wheels in $\mathcal{W}$** .

VI. GPU ACCELERATION

The proposed LIO framework contains several computationally intensive modules that are highly amenable to GPU parallelization. Consequently, GPU acceleration can be exploited to improve the real-time performance of the overall system. In particular, two modules benefit the most from GPU implementation:

- 1) **Evaluation of RBF-related terms.** The evaluation of the RBF terrain model involves computing exponential terms for a large number of center–point pairs. This operation is highly parallelizable and is therefore offloaded to the GPU to exploit data-level parallelism. In our implementation, both the matrix–vector multiplication in the terrain height estimation (11) and the entries of the RBF kernel matrix in (20a) are computed in parallel on the GPU. In addition, evaluating the manifold soft constraints requires both the residuals in (31) for **all wheel contacts $j \in \mathcal{W}$** and the corresponding Jacobian blocks in (35). These computations repeatedly involve evaluating the RBF terrain function and its spatial derivatives at the contact points. Since each term in the corresponding summations is independent, the contributions in the residual and Jacobian evaluations, including those in (37)–(42), can also be computed in parallel on the GPU.
- 2) **The Sparse kernel matrix inversion.** Although the Gaussian RBF is not compactly supported, its value decays rapidly with distance. Combined with the active-center approximation introduced in Sec. IV-C, this leads to a numerically sparse kernel matrix $\mathcal{H}_k$ in practice. Therefore, the associated matrix inversion and linear algebra operations can be accelerated using the `cuSOLVER` library from the CUDA Toolkit, which provides efficient sparse linear algebra routines on the GPU. In our implementation, the active-block matrix inversion and the related linear algebra operations appearing in (21), (22), and (23) are all accelerated on the GPU.

VII. EXPERIMENTS

Except for the cross-platform experiment in Sec. VII-G, all experiments are conducted on a LimX Dynamics Tron1 two

legged-wheel robot operating in off-road locomotion mode. We add a sensor kit to Tron1, which includes an inverted Mid360 LiDAR and its built-in IMU, a GPS+RTK module, and joint sensors, as shown in Fig. 4. The Mid360 provides 10-Hz point clouds and 200-Hz IMU measurements, while the joint sensors provide 500-Hz joint-angle measurements. The GPS+RTK module provides 10-Hz position references for the six quantitatively evaluated Tron1 sequences. The inverted LiDAR increases the density of ground observations for terrain reconstruction, but it is a sensing configuration rather than a prerequisite of the proposed formulation. The Tron1 experiments are processed on a laptop equipped with an Intel i5-12490F CPU, 32 GB RAM, and an NVIDIA RTX 3070 GPU.

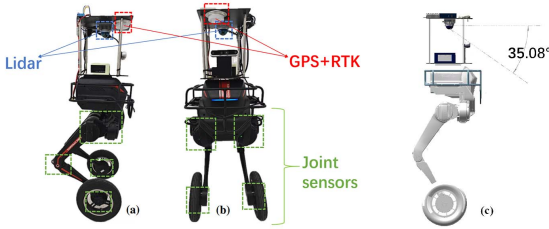


Fig. 4. Tron1 platform and sensor configuration used in the Tron1 experiments: an inverted Mid360 LiDAR with its built-in IMU, a GPS+RTK module, and joint sensors. (a) Side view, (b) front view, and (c) LiDAR field of view.

A. The Dataset



Fig. 5. Dataset scenarios: (a) Staircase Scene, (b) Artificial Hill, (c) Rose Garden, (d) Botanical Garden, (e) Indoor Staircase Only and (f) Wet Grass. These datasets cover diverse terrain elevations. Scenarios (a)–(e) contain distinct stair structures for evaluating z -axis localization under abrupt elevation changes. The Wet Grass sequence provides a complementary slippery-surface stress test on wet grass terrain with an undulating overall elevation profile and local surface irregularities.

Due to the lack of publicly available datasets for legged-wheel robots, we collect [seven](#) datasets in real-world unstructured scenarios that cover a wide variety of indoor and outdoor terrains, surface textures, and elevation profiles. More specifically, these datasets include abrupt elevation changes, mixed feature densities, and both structured and unstructured environments. The scenarios are as follows:

- **Staircase Scene 1 & 2:** These two sequences traverse flat pavements and different staircases with step heights

ranging from 5 cm to 15 cm and 1–8 steps, including both single-pass and repeated back-and-forth traversals.

- **Artificial Hill:** This sequence traverses stone-slab paths with multiple single-step stairs, starting from the foot of the hill to the summit, and then returning to the starting point along a different route. It exhibits substantial elevation changes and diverse terrain surface textures.
- **Rose Garden:** This sequence traverses short staircases, flat pavements, and grass slopes, with low lateral obstacles such as flower beds and shrubs. In addition, some staircases are flanked by low walls, which result in sparse geometric features around the staircase region.
- **Botanical Garden:** This sequence covers a long route in the botanical garden, including a suspension bridge, an arch bridge, multiple single-step stairs, and a stone-slab path.
- **Indoor Staircase Only:** This sequence contains a short indoor traversal in which the robot climbs a long staircase. It is used as a qualitative stress test and does not provide RTK ground truth.
- **Wet Grass:** This sequence is collected after rain on wet grass terrain with an undulating overall elevation profile and local surface irregularities, and serves as a slippery-surface stress test.

Ground Truth: For each sequence (except for “Indoor Staircase Only”), we additionally record an RTK-corrected position trajectory, including latitude, longitude, and altitude, using the GPS+RTK system. This provides centimeter-level 3D reference trajectories for quantitative evaluation. We use the RTK-corrected GPS positions as the ground-truth trajectory. However, since the GPS+RTK system provides only latitude, longitude, and altitude, but not orientation, we evaluate only the translation errors of the compared methods.

B. RBF-based Terrain Approximation Result

In our experiments, we set the mesh resolution of the RBF centers to 0.07 m, the bandwidth parameter σ to 0.04 m, and the measurement noise level σ_ϵ to 0.1. The mesh resolution determines the number of RBF centers and affects both approximation fidelity and computational cost. Empirically, a resolution of 0.07 m provides sufficient geometric detail for typical terrain variations while maintaining real-time efficiency. The bandwidth parameter σ determines the spatial influence of each kernel, while the noise level σ_ϵ accounts for sensor uncertainty and improves robustness to noisy point clouds.

In addition, the ridge penalty parameter λ in ridge regression must be chosen carefully. A too small λ may lead to noise-sensitive and numerically unstable fitting, whereas a too large λ may over-smooth the terrain and blur local geometric details. [An external ground-truth terrain model is difficult to obtain.](#) We therefore evaluate the fitted surface against the LiDAR-observed terrain points for different values of λ . Each point $\mathbf{p}_k^i = [\mathbf{x}_k^i, p_{z,k}^i]^\top$ is [expressed in the world frame](#), and its [point-wise absolute vertical residual](#) is defined as

$$e_k^i = |p_{z,k}^i - f(\mathbf{x}_k^i; \hat{\mathbf{w}}_k^{RG})|. \quad (43)$$



Fig. 6. Boxplots of the terrain-approximation error under different RBF parameter settings on the Rose Garden sequence: (a) ridge penalty λ , with center resolution 0.07 m and bandwidth 0.04 m; (b) center-resolution/bandwidth pairs, with $\lambda = 0.01$.

In particular, only the points whose horizontal coordinates lie inside the current RBF region of interest are included. For each tested value of λ , we pool the residuals over all valid terrain points and processed LiDAR frames in the Rose Garden sequence. Since there are LiDAR outliers caused by water-puddle reflections, dynamic objects, and local non-terrain returns, we exclude the top 1% of the pooled residuals when computing the error statistics. Fig. 6(a) shows a broad error distribution for $\lambda = 0.001$ and more high-error outliers for $\lambda = 0.1$ despite its lower median. We therefore choose $\lambda = 0.01$ to limit large residuals while maintaining a compact distribution in all subsequent experiments.

To further investigate the joint influence of the RBF center mesh resolution and bandwidth, we compare three representative parameter configurations on the Rose Garden sequence. As shown in Fig. 6(b), the configuration with a center mesh resolution of 0.07 m and a bandwidth of 0.04 m yields a median error comparable to that of the coarser configuration, while achieving the smallest interquartile range and the shortest upper whisker. The finer configuration allows more candidate centers and more localized basis functions, which increase local flexibility but can also increase sensitivity to sparse or noisy terrain observations. In contrast, the coarser configuration combines fewer candidate centers with wider basis functions, reducing its ability to represent local terrain variations. The selected configuration therefore provides the best empirical balance between local representation capability and spatial smoothness among the tested settings and is used in all subsequent experiments.

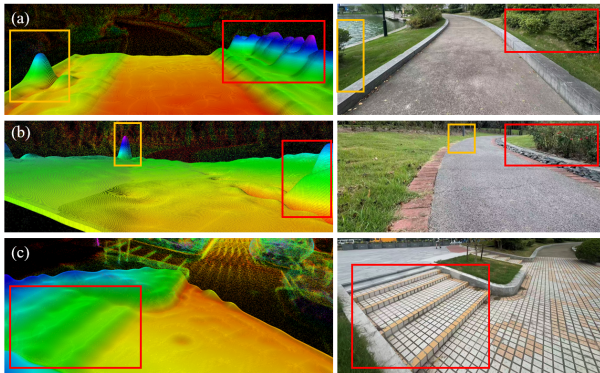


Fig. 7. RBF-based terrain approximation in different environments: (a) flat area, (b) garden pathway, and (c) staircase scene. The colored boxes indicate corresponding objects or local structures across the results.

Next, we qualitatively demonstrate the RBF-based terrain

approximation on three representative terrains in Fig. 7. In the flat-area scene shown in Fig. 7(a), the proposed method reconstructs broad planar regions together with local structures such as bushes and flower beds. In the garden-pathway scene shown in Fig. 7(b), the reconstructed terrain captures the local slope variations, the utility pole on the slope, and the boundary between the pathway and the adjacent grass field. In the staircase scene shown in Fig. 7(c), the method preserves multiple discrete elevation changes corresponding to individual stair steps.

To further quantify the approximation accuracy, we report the error histogram of the RBF-based terrain approximation in the remaining scenarios of the dataset, as shown in Fig. 8(d). The histogram is constructed by pooling all valid e_k^i over all terrain points and processed frames in these sequences; the largest 1% of the pooled residuals are removed before the displayed statistics are computed. As a result, under the same evaluation setup, 94.70% of the errors fall below 0.05 m. This indicates that the proposed RBF-based terrain approximation achieves good accuracy on terrains with stairs and grass surfaces. This observation is also consistent with the improved localization performance shown later in Fig. 10, where RBF-LIO reduces the vertical localization drift that is visible in Fast-LIO2.

Moreover, to demonstrate the real-time performance of the proposed method, we report the computation-time histograms of the main modules in Fig. 8(a)–(c). The overall framework runs at 10 Hz, which satisfies the real-time requirement of the LiDAR input stream.

C. Ablation Study

We conduct three ablation studies to evaluate the terrain manifold constraint, adaptive RBF center selection, and GPU acceleration. The manifold constraint is evaluated on the five main quantitative sequences, whereas center selection and GPU acceleration are evaluated on the Rose Garden sequence, which contains mixed terrain surfaces and representative height variations. In the manifold ablation, the full and w/o-manifold variants use identical LiDAR, IMU, and joint measurements, initialization, feature extraction, scan-matching implementation, and parameters; the only difference is whether the manifold residuals are included.

As shown in Table I, removing the terrain manifold constraint causes a substantial degradation in both overall and vertical localization accuracy on all evaluated sequences. This indicates that the differentiable terrain manifold reduces overall trajectory error and vertical drift during abrupt elevation

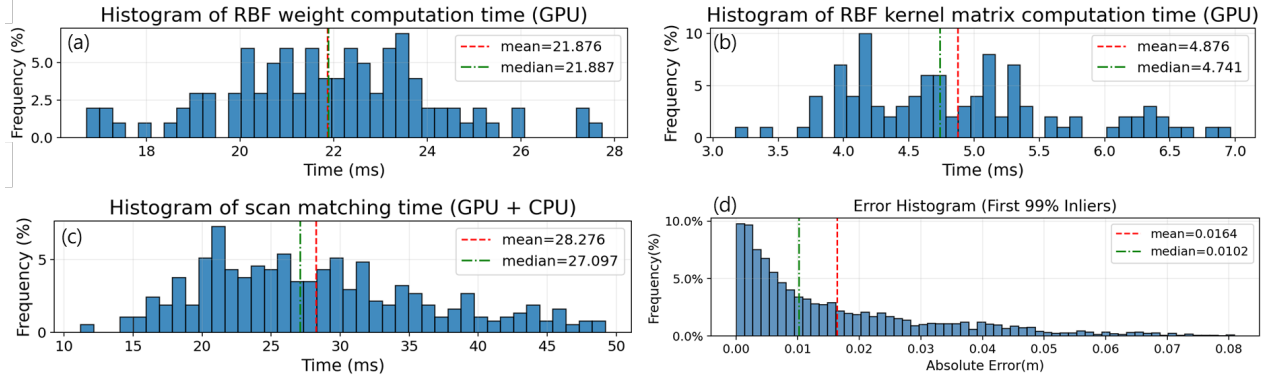


Fig. 8. (a) Histogram of the computation time for RBF weight updates on the GPU. (b) Histogram of the computation time for RBF kernel-matrix evaluation on the GPU. (c) Histogram of the computation time for scan matching on the GPU and CPU. (d) Histogram of the terrain-approximation error on the remaining scenarios in the dataset after excluding the top 1% outliers.

TABLE I

ABLATION STUDY OF THE TERRAIN MANIFOLD CONSTRAINT. VALUES DENOTE ATE RMSE IN METERS, AND THE VALUES IN PARENTHESES DENOTE THE CORRESPONDING z -AXIS ATE RMSE.

Sequence	Full RBF-LIO	w/o manifold
Staircase 1	0.6752 (0.0617)	0.8457 (0.1751)
Staircase 2	0.7858 (0.0625)	1.0147 (0.1738)
Artificial Hill	0.4759 (0.1354)	0.7853 (0.2379)
Rose Garden	0.5824 (0.1034)	0.7987 (0.2256)
Botanical Garden	0.9491 (0.2147)	1.1648 (0.3276)

TABLE II

ABLATION STUDY OF THE ADAPTIVE RBF CENTER SELECTION STRATEGY ON THE ROSE GARDEN SEQUENCE. THE COMPUTATION TIME IS THE SUM OF KERNEL MATRIX CONSTRUCTION TIME AND WEIGHT UPDATE TIME.

Variant	Active centers / ratio	Mean error	Computation time
Adaptive centers	883.8 / 39.9%	0.0167 m	26.64 ms
w/o adaptive centers	2215.1 / 100.0%	0.0131 m	105.15 ms

changes.

Next, we evaluate adaptive RBF center selection on the Rose Garden sequence. Both variants use identical terrain observations and RBF parameters. The proposed strategy activates a candidate center only when it is supported by nearby LiDAR terrain points, whereas the ablated variant activates all candidate centers within the current region of interest.

As shown in Table II, adaptive center selection retains 39.9% of the candidate centers and reduces the RBF computation time from 105.15 ms to 26.64 ms. The mean error increases only slightly from 0.0131 m to 0.0167 m. Thus, adaptive center selection substantially improves online efficiency while preserving comparable terrain-approximation accuracy.

Finally, we evaluate GPU acceleration on the Rose Garden sequence. The GPU and CPU variants use identical terrain observations and RBF parameters and perform the same RBF computations; only the execution backend differs. The GPU variant uses CUDA for kernel-matrix construction, weight solving, and terrain-height evaluation, whereas the CPU variant performs these operations on the CPU.

TABLE III

ABLATION STUDY OF GPU ACCELERATION FOR ONLINE RBF TERRAIN APPROXIMATION ON THE ROSE GARDEN SEQUENCE. THE TOTAL COMPUTATION TIME IS THE SUM OF KERNEL MATRIX CONSTRUCTION TIME, WEIGHT SOLVING TIME, AND TERRAIN EVALUATION TIME.

Variant	Mean error	Kernel	Weight	Evaluation	Total
GPU	0.0167 m	5.06 ms	21.58 ms	6.53 ms	33.17 ms
CPU	0.0165 m	98.67 ms	4339.21 ms	69.04 ms	4506.92 ms

As shown in Table III, the two variants achieve comparable terrain-approximation accuracy. GPU acceleration reduces the total RBF computation time from 4506.92 ms to 33.17 ms, corresponding to a $135.9\times$ speedup. The GPU runtime is well below the 100 ms frame interval of the 10 Hz LiDAR, demonstrating that GPU parallelization enables online RBF terrain reconstruction at the sensor rate.

D. Robustness to Reduced Terrain Observations

To evaluate the robustness of RBF-LIO to the density and angular coverage of LiDAR observations, we conduct a controlled stress test on the Rose Garden sequence. Four settings are considered: the original full LiDAR point cloud, uniform random subsampling to 75% and 50% of the points in each frame, and frame-wise random sector occlusion. For sector occlusion, a contiguous 30° azimuthal sector is sampled uniformly from the LiDAR field of view in each frame, and all points within that sector are removed. The resulting point clouds are used for both recursive RBF updating and scan matching; all other sensor measurements, initialization, system parameters, and evaluation settings remain unchanged.

As shown in Table IV, all three reduced-observation settings produce only slight increases in the mean terrain-approximation error, while the ATE, z -ATE, and RTE remain close to those under full observations. These results demonstrate the robustness of the proposed RBF-LIO to sparse LiDAR measurements and occlusions.

E. Comparison Results

Our proposed RBF-LIO is evaluated against five representative baselines: Fast-LIO2 [2], ROLO-SLAM [13], Leg-

TABLE IV
LOCALIZATION RESULTS UNDER REDUCED LiDAR OBSERVATIONS ON THE ROSE GARDEN SEQUENCE. ATE, z -ATE, AND RTE ($\Delta t = 0.1$ s) ARE REPORTED IN METERS.

Setting	Kept ratio	Mean error	ATE (z -ATE)	RTE
Full	100.00%	0.0167 m	0.5824 (0.1034)	0.0363
Sparse 75%	75.01%	0.0174 m	0.5860 (0.1120)	0.0364
Sparse 50%	50.00%	0.0177 m	0.5867 (0.1184)	0.0368
Sector Occlusion 30°	91.58%	0.0169 m	0.5841 (0.1104)	0.0363

KILO [9], GLIM [4], and Wheel-Legged SLAM [10]. In particular, Fast-LIO2 and GLIM have demonstrated strong localization and mapping performance on ground vehicles and aerial platforms. In contrast, ROLO-SLAM, Leg-KILO, and Wheel-Legged SLAM are more closely related to our setting, since they are designed to mitigate z -axis pose drift. Although Leg-KILO and Wheel-Legged SLAM were originally developed for quadruped robots, their core mechanisms of incorporating kinematic constraints into LiDAR-inertial odometry are general. To ensure a fair comparison, we adapt these baselines to our legged-wheel platform. Specifically, we provide the requisite leg-joint kinematics for both Leg-KILO and Wheel-Legged SLAM. In addition, the contact indicator used by Leg-KILO's kinematic/contact-height update is kept active to reflect continuous wheel-terrain contact.

To make the comparison protocol explicit, Table V summarizes the sensor measurements and additional geometric information used by each method. We do not modify general-purpose LiDAR/LiDAR-IMU odometry baselines (Fast-LIO2, GLIM, and ROLO-SLAM) to use joint measurements or terrain contact information, because doing so would change their original algorithmic assumptions and require designing new factors that are outside the scope of those methods. In addition, Leg-KILO and Wheel-Legged SLAM are used as closely related kinematic/contact-aware baselines.

In addition, we keep the following settings the same in all comparisons:

- **Sensor measurements:** All methods use the same Mid360 LiDAR scans (10 Hz). RBF-LIO, Fast-LIO2, GLIM, Leg-KILO and Wheel-Legged SLAM use the IMU measurements (200 Hz). In addition, RBF-LIO, Leg-KILO and Wheel-Legged SLAM also use the joint-sensor measurements (500 Hz).
- **System parameters:** Across all methods, we maintain a consistent voxel resolution of 0.2 m for down-sampling in scan matching. For the IMU configuration, the noise covariances for linear acceleration and angular velocity are set to $10^{-2}\mathbf{I}_3$, while their respective bias covariances are set to $10^{-4}\mathbf{I}_3$.

All methods are evaluated over the same trajectory intervals using the same RTK reference, time association, and alignment protocol.

We first examine the indoor staircase-only sequence as a stress test for abrupt elevation changes. As shown in Fig. 9, RBF-LIO reconstructs a structurally consistent staircase map. In contrast, the other methods exhibit different degrees of ghosting, layering, or staircase-surface distortion, indicating

reduced robustness when repeated height transitions dominate the scene. This result directly illustrates the benefit of explicitly incorporating terrain geometry into the odometry optimization.

We further evaluate the mapping results on the outdoor sequences in Fig. 10. The improvements are most visible along the vertical dimension. In Rose Garden, where sparse features and abrupt height changes occur simultaneously, Fast-LIO2 suffers from map layering, while Wheel-Legged SLAM fails near the staircase region. In Botanical Garden, Leg-KILO shows noticeable drift on complex terrain. These results reflect the limitations of scan matching alone, local contact constraints, and local-plane-derived scalar height updates that do not maintain a shared nonplanar terrain function for contact constraints. In contrast, RBF-LIO uses the differentiable RBF terrain manifold to regularize the height estimate, resulting in more globally consistent maps.

Finally, Fig. 11 compares the trajectories on Rose Garden. This sequence is selected because it contains sparse features, grass slopes, and short staircases. Fast-LIO2 and Wheel-Legged SLAM diverge, while ROLO-SLAM produces a large pose error; therefore, these three trajectories are not shown. Leg-KILO and GLIM still exhibit z -axis drift, whereas RBF-LIO remains closest to the RTK reference.

Next, we quantitatively compare RBF-LIO with the baselines using ATE, z -ATE and RTE, as reported in Tables VI and VII. The best results are highlighted in bold. Both GLIM and RBF-LIO successfully estimate poses on all evaluated sequences. On feature-rich sequences with abrupt height changes, such as Staircase Scene 1, Staircase Scene 2, and Artificial Hill, RBF-LIO achieves overall ATE values comparable to the strongest baselines while maintaining lower or competitive z -axis errors. On more challenging sequences such as Rose Garden and Botanical Garden, the advantage of RBF-LIO becomes more evident, especially in suppressing accumulated vertical drift. RBF-LIO also outperforms ROLO-SLAM, which is explicitly designed to reduce vertical errors.

The improvement in RTE is less pronounced than that in ATE because RTE measures short-horizon local consistency, whereas ATE reflects accumulated trajectory-wide drift. Since the proposed manifold constraint mainly takes effect during height transitions, its benefit is more clearly reflected in ATE and z -ATE. Overall, the qualitative and quantitative results show that RBF-LIO improves robustness under abrupt elevation changes while remaining competitive on normal terrain.

F. Wet-Grass Stress Test

To evaluate RBF-LIO under slippery-surface conditions, we collect an additional wet-grass sequence after rain and compare RBF-LIO with Fast-LIO2, Leg-KILO, GLIM, and Wheel-Legged SLAM.

As shown in Table VIII, RBF-LIO achieves the lowest ATE, z -ATE, and RTE among the evaluated methods. These results demonstrate that RBF-LIO maintains overall and vertical localization accuracy as well as short-term odometry stability in this wet-grass test. We conjecture that the RBF surface captures the overall terrain variation despite local deformation and thus still provides useful geometric correction to LIO.

TABLE V
SENSOR MEASUREMENTS AND ADDITIONAL GEOMETRIC INFORMATION USED BY EACH METHOD IN THE COMPARISON.

Method	LiDAR	IMU	Joint sensors	Kinematic/contact information	Explicit terrain model
Fast-LIO2 [2]	Yes	Yes	No	No	No
GLIM [4]	Yes	Yes	No	No	No
ROLO-SLAM [13]	Yes	No	No	No	No
Leg-KILO [9]	Yes	Yes	Yes	Leg kinematics and local-plane-based contact-height detection	Local plane for contact-height projection
Wheel-Legged SLAM [10]	Yes	Yes	Yes	Wheel-leg kinematics	No
RBF-LIO (ours)	Yes	Yes	Yes	Wheel-terrain contact geometry	Online RBF terrain manifold

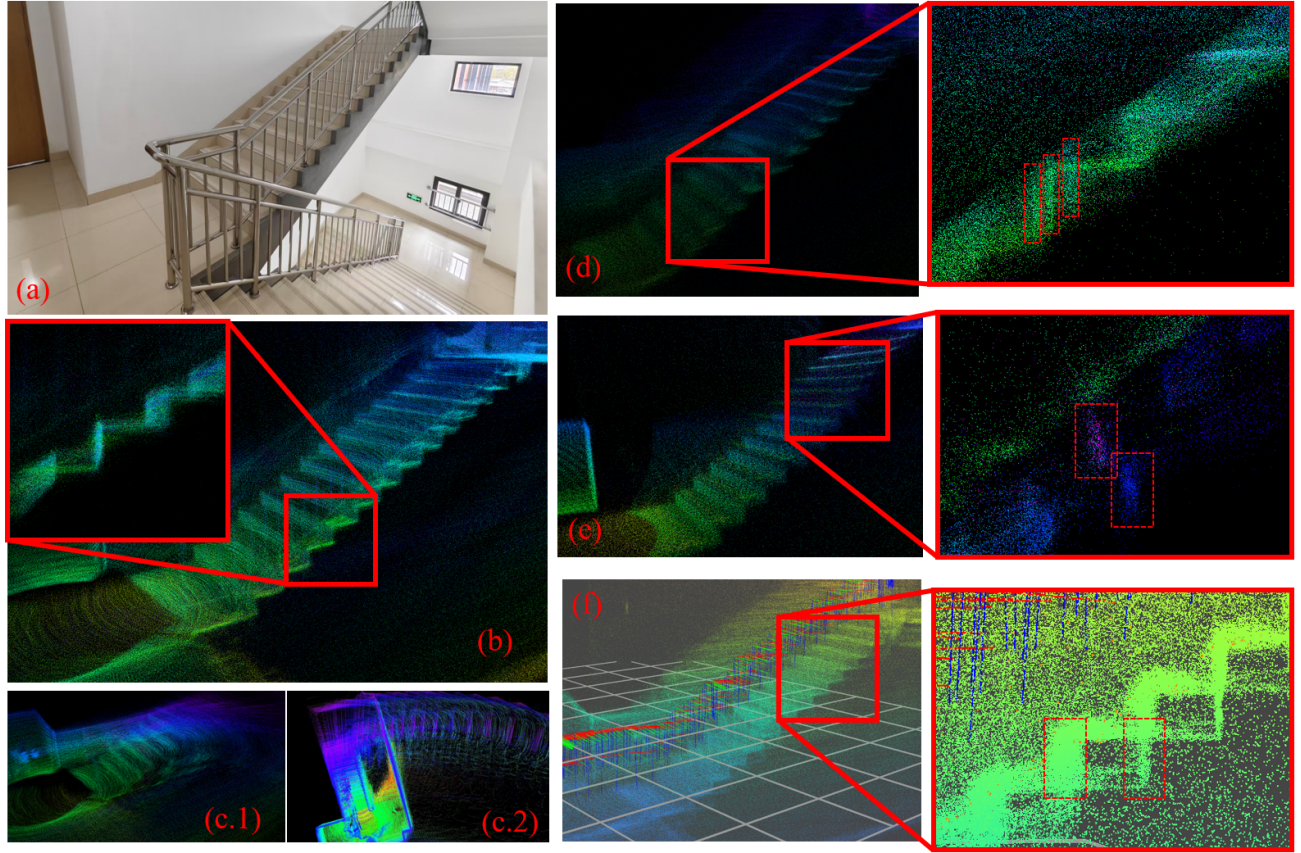


Fig. 9. Qualitative comparison of point cloud maps generated in an indoor staircase-only scenario. (a) Reference photograph of the environment. Mapping results produced by (b) RBF-LIO, (c.1) Wheel-Legged SLAM, (c.2) top view of Wheel-Legged SLAM, (d) FAST-LIO2, (e) Leg-KILO, and (f) GLIM. The zoomed-in views highlight the staircase structure reconstructed by each method. The dashed red rectangles indicate the same vertical plane of a single step for visual comparison.

TABLE VI
ATE [20] (m) FOR DIFFERENT METHODS. VALUES IN PARENTHESES DENOTE z -AXIS ERRORS. A SLASH DENOTES FAILURE.

Sequence	RBF-LIO		Fast-LIO2 [2]		ROLO-SLAM [13]		Leg-KILO [9]		GLIM [4]		Wheel-Legged SLAM [10]	
	RMSE	max	RMSE	max	RMSE	max	RMSE	max	RMSE	max	RMSE	max
Staircase 1	0.6752 (0.0617)	1.3126 (0.3588)	0.7471 (0.0587)	1.4464 (0.3166)	21.6712 (1.0336)	29.2766 (2.4170)	0.6347 (0.0615)	1.1855 (0.3393)	0.7357 (0.1793)	1.3265 (0.4787)	1.4982 (0.3200)	2.5845 (0.7898)
Staircase 2	0.7858 (0.0625)	1.2764 (0.3372)	0.9039 (0.0670)	1.5034 (0.3207)	36.7933 (3.8083)	65.8538 (9.8349)	0.8160 (0.0832)	1.5232 (0.3841)	0.9872 (0.2074)	1.7557 (0.5310)	1.5265 (0.2032)	3.6606 (0.5622)
Artificial Hill	0.4759 (0.1354)	0.9555 (0.5144)	0.6088 (0.1553)	1.1251 (0.5210)	16.1785 (6.4400)	24.1648 (11.2143)	0.4534 (0.1394)	0.8362 (0.5162)	0.6190 (0.2300)	1.1597 (0.6370)	1.8872 (0.3987)	3.3049 (1.0807)
Rose Garden	0.5824 (0.1034)	1.0972 (0.2628)	/	/	16.9995 (1.5915)	27.4158 (4.8372)	0.9069 (0.1669)	1.8861 (0.6708)	1.2513 (0.0965)	2.1290 (0.3550)	/	/
Botanical Garden	0.9491 (0.2147)	1.6648 (0.4915)	1.2962 (0.3683)	4.0772 (1.2227)	22.3889 (3.3272)	41.6459 (7.2956)	/	/	0.9980 (0.1595)	1.8093 (0.4791)	2.6237 (0.3509)	4.5260 (0.8286)

TABLE VII

RTE [20] (m) ($\Delta t = 0.1$ s) FOR DIFFERENT METHODS. VALUES IN PARENTHESES DENOTE z -AXIS ERRORS. SLASH MEANS FAILURE.

Sequence	RBF-LIO		Fast-LIO2 [2]		ROLO-SLAM [13]		Leg-KILO [9]		GLIM [4]		Wheel-Legged SLAM [10]	
	RMSE	max	RMSE	max	RMSE	max	RMSE	max	RMSE	max	RMSE	max
Staircase 1	0.0503	0.2267	0.0556	0.2546	0.1694	1.1933	0.0478	0.2344	0.0545	0.4161	0.1073	0.7914
Staircase 2	(0.0170)	(0.1333)	(0.0124)	(0.1058)	(0.0401)	(0.3205)	(0.0124)	(0.1064)	(0.0232)	(0.4037)	(0.0108)	(0.0968)
	0.0493	0.2227	0.0493	0.2333	0.2027	0.8776	0.0461	0.2294	0.0480	0.2257	0.0869	0.8077
Artificial Hill	(0.0138)	(0.0758)	(0.0148)	(0.1006)	(0.0650)	(0.3859)	(0.0141)	(0.1095)	(0.0140)	(0.1025)	(0.0113)	(0.0779)
	0.0277	0.4614	0.0864	0.5699	0.3977	8.7865	0.0605	0.4617	0.0624	0.4645	0.1233	1.1333
Rose Garden	(0.0314)	(0.4198)	(0.0365)	(0.4227)	(0.0993)	(0.7038)	(0.0301)	(0.4434)	(0.0305)	(0.4478)	(0.0336)	(0.4536)
	0.0363	0.1744	/	/	0.1881	0.6960	0.0595	0.2337	0.0413	0.2288	/	/
Botanical Garden	(0.0154)	(0.0979)			(0.0530)	(0.2968)	(0.0114)	(0.0860)	(0.0113)	(0.1415)		
	0.0849	0.3815	0.0664	0.5346	0.2104	0.8683	/	/	0.0575	0.3550	0.0938	0.7006
	(0.0179)	(0.3343)	(0.0197)	(0.3514)	(0.0639)	(0.3172)			(0.0185)	(0.3472)	(0.0168)	(0.3530)

TABLE VIII

WET-GRASS STRESS TEST UNDER A SLIPPERY-SURFACE CONDITION. ATE, z -ATE, AND RTE ($\Delta t = 0.1$ s) ARE REPORTED IN METERS.

Method	ATE	z -ATE	RTE
Fast-LIO2	0.8836	0.2248	0.0689
Leg-KILO	0.7911	0.1712	0.0590
GLIM	0.9022	0.2045	0.0664
Wheel-Legged SLAM	1.2837	0.3603	0.1081
RBF-LIO (ours)	0.7023	0.1442	0.0557

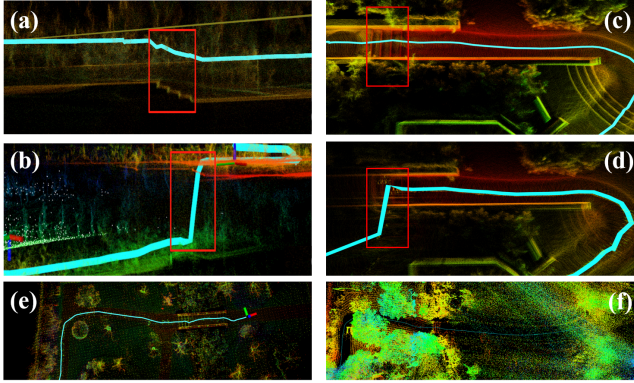


Fig. 10. Visualization of the mapping results: (a) RBF-LIO on the Rose Garden dataset, (b) Fast-LIO2 on the Rose Garden dataset, (c) RBF-LIO on a staircase in a feature-rich environment, (d) Wheel-Legged SLAM in the same environment as (c), (e) RBF-LIO on the Botanical Garden dataset, and (f) Leg-KILO on the Botanical Garden dataset.

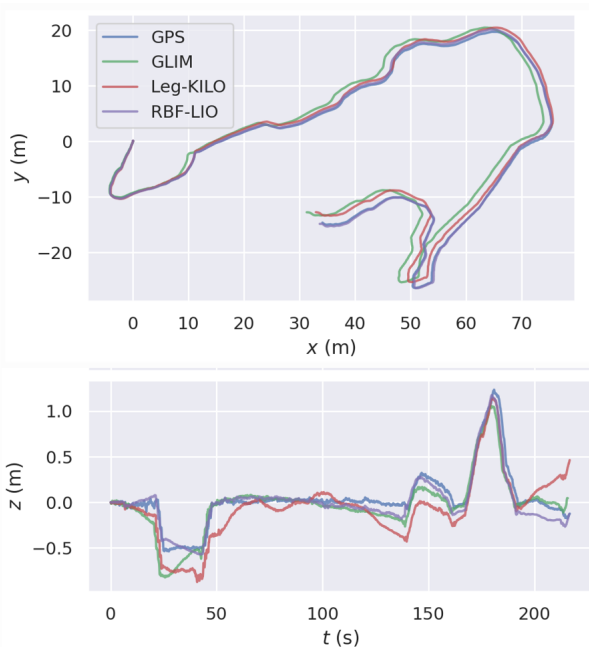


Fig. 11. Trajectories of different methods on the Rose Garden dataset: projection on the xy -plane and evolution along the z -axis.

G. Cross-Platform Staircase Stress Test

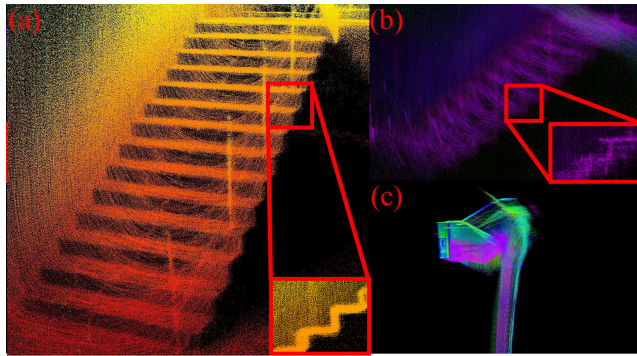
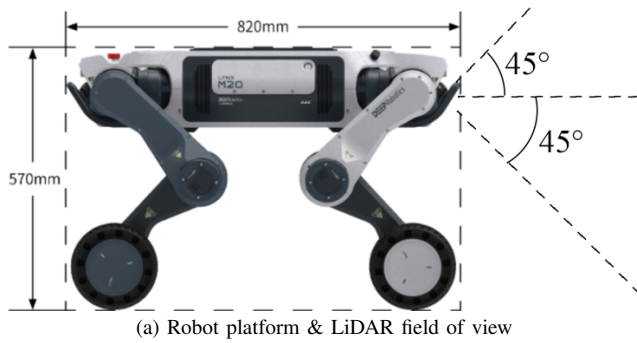
To evaluate RBF-LIO under a different robot platform and LiDAR mounting configuration, we conduct a qualitative cross-platform staircase test on a DeepRobotics Lynx M20 four-wheel-legged robot. Unlike the inverted LiDAR used in the Tron1 experiments, the M20 LiDAR faces forward, with a vertical viewing sector of $\pm 45^\circ$ about the forward direction, as illustrated in Fig. 12(a). The selected half-floor staircase segment contains 183 poses over 36.40 s, with a trajectory length of 12.03 m and an elevation range of 2.23 m. Since GPS+RTK ground truth is unavailable on the M20, we report qualitative mapping results rather than quantitative trajectory errors.

The full RBF-LIO framework is used in this experiment with $\mathcal{W} = \{\text{FL}, \text{FR}, \text{RL}, \text{RR}\}$. The wheel centers and wheel-axis directions are evaluated using the M20 kinematic model, while the contact points remain optimization variables. The manifold residual and cost retain the unified formulations in (31) and (33). The remaining settings for terrain modeling, center selection, GPU acceleration, and scan matching are kept consistent with the main experiments.

As shown in Fig. 12, RBF-LIO preserves distinct stair steps and a consistent staircase structure on the M20 platform. In contrast, Leg-KILO produces a blurred and thick staircase map, while Wheel-Legged SLAM fails to preserve a consistent staircase structure and exhibits severe trajectory drift. This comparison demonstrates that the unified RBF-LIO framework remains effective on a different wheel-legged platform with a different LiDAR mounting configuration.

VIII. CONCLUSION AND FUTURE WORK

In this paper, we propose RBF-LIO, which recursively updates an RBF terrain model and incorporates its manifold into



(b) Side and zoomed-in mapping results. From left to right, the zoomed-in maps are produced by RBF-LIO, Leg-KILO, and Wheel-Legged SLAM.

Fig. 12. Qualitative cross-platform staircase stress test on the M20 four-wheel-legged robot. In the zoomed-in comparison, RBF-LIO preserves a clearer staircase structure, Leg-KILO produces a blurred and thick map, and Wheel-Legged SLAM suffers from severe drift.

LIO optimization as soft wheel-terrain constraints to improve vertical localization accuracy. We also release a dataset for legged-wheel SLAM research.

Experiments demonstrate that RBF-LIO suppresses z -axis pose drift on terrains with large and abrupt elevation changes compared to state-of-the-art methods. Ablation studies further confirm that the manifold constraint improves overall and vertical localization accuracy, while adaptive center selection and GPU acceleration make online RBF terrain modeling computationally practical. Reduced-observation tests confirm robustness to sparse ground observations.

RBF-LIO assumes stable contact, one effective contact point per wheel, and a locally single-valued terrain surface. Near sharp edges, the contact point may be ambiguous; wheel slip, contact loss, vibration, or non-single-valued terrain may cause incorrect pose corrections. These cases are not explicitly detected. Nevertheless, RBF-LIO still outperforms the baselines in the slippery wet-grass and staircase tests. We therefore conjecture that the RBF surface captures overall terrain variation and continues to provide useful geometric correction to LIO. Future work will estimate contact reliability, down-weight low-reliability constraints, and evaluate complex non-smooth terrain and quantitative cross-platform performance.

REFERENCES

- [1] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and D. Rus, "Lio-sam: Tightly-coupled lidar inertial odometry via smoothing and mapping," in *2020 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2020, pp. 5135–5142.
- [2] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "Fast-lio2: Fast direct lidar-inertial odometry," *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 2053–2073, 2022.
- [3] W. Xu and F. Zhang, "Fast-lio: A fast, robust lidar-inertial odometry package by tightly-coupled iterated kalman filter," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3317–3324, 2021.
- [4] K. Koide, M. Yokozuka, S. Oishi, and A. Banno, "Glim: 3d range-inertial localization and mapping with gpu-accelerated scan matching factors," *Robotics and Autonomous Systems*, vol. 179, p. 104750, 2024.
- [5] P. Fankhauser, M. Bloesch, C. Gehring, M. Hutter, and R. Siegwart, "Robot-centric elevation mapping with uncertainty estimates," in *International Conference on Climbing and Walking Robots (CLAWAR)*, 2014.
- [6] P. Fankhauser, M. Bloesch, and M. Hutter, "Probabilistic terrain mapping for mobile robots with uncertain localization," *IEEE Robotics and Automation Letters (RA-L)*, vol. 3, no. 4, pp. 3019–3026, 2018.
- [7] T. Miki, L. Wellhausen, R. Grandia, F. Jenelten, T. Homberger, and M. Hutter, "Elevation mapping for locomotion and navigation using gpu," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022, pp. 2273–2280.
- [8] W. Tian, J. Wang, P. Yang, W. Xiao, and S. Zlatanova, "Lio-gc: Lidar inertial odometry with adaptive ground constraints," *Remote Sensing*, vol. 17, no. 14, p. 2376, 2025.
- [9] G. Ou, D. Li, and H. Li, "Leg-kilo: Robust kinematic-inertial-lidar odometry for dynamic legged robots," *IEEE Robotics and Automation Letters*, 2024.
- [10] Y. Wen, Y. Li, Q. Shang, C. Jiang, H. Hou, H. Liu, Y. Zhang, and L. Han, "Wheel-legged slam: Indoor lidar-inertial slam integrating kinematic model of wheel-legged robots," *IEEE Robotics and Automation Letters*, vol. 10, no. 2, pp. 1273–1280, 2024.
- [11] D. Wisth, M. Camurri, and M. Fallon, "Vilens: Visual, inertial, lidar, and leg odometry for all-terrain legged robots," *IEEE Transactions on Robotics*, vol. 39, no. 1, pp. 309–326, 2022.
- [12] Q. Zhao, M. Li, Y. Shi, X. Chen, Z. Yu, L. Han, Z. Fu, J. Zhang, C. Li, Y. Zhang *et al.*, "Liko: Lidar, inertial, and kinematic odometry for bipedal robots," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 1180–1185.
- [13] Y. Wang, B. Ren, X. Zhang, P. Wang, C. Wang, R. Song, Y. Li, and M. Q.-H. Meng, "Rolo-slam: rotation-optimized lidar-only slam in uneven terrain with ground vehicle," *Journal of Field Robotics*, vol. 42, no. 3, pp. 880–902, 2025.
- [14] J. Li, C. Li, H. Chen, H. Guo, X. Luo, C. P. Chen, and C. Yang, "Ars-slam: Accurate robust spinning lidar slam for a quadruped robot in large-scale scenario," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 2635–2641.
- [15] C. Zhang, Z. Yang, Q. Fang, C. Xu, H. Xu, X. Xu, and J. Zhang, "Frl-slam: A fast, robust and lightweight slam system for quadruped robot navigation," in *2021 IEEE International Conference on Robotics and Biomimetics (ROBIO)*. IEEE, 2021, pp. 1165–1170.
- [16] A. Leininger, M. Ali, H. Jardali, and L. Liu, "Gaussian process-based traversability analysis for terrain mapless navigation," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 10925–10931.
- [17] J. C. Carr, R. K. Beatson, J. B. Cherrie, T. J. Mitchell, W. R. Fright, B. C. McCallum, and T. R. Evans, "Reconstruction and representation of 3d objects with radial basis functions," in *Proceedings of the 28th annual conference on Computer graphics and interactive techniques*, 2001, pp. 67–76.
- [18] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.
- [19] J. J. Moré, "The levenberg-marquardt algorithm: implementation and theory," in *Numerical analysis: proceedings of the biennial Conference held at Dundee, June 28–July 1, 1977*. Springer, 2006, pp. 105–116.
- [20] Z. Zhang and D. Scaramuzza, "A tutorial on quantitative trajectory evaluation for visual (-inertial) odometry," in *2018 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2018, pp. 7244–7251.